

Specification Testing Methodology

Motivation

Traditional software engineering treats specifications as requirements documents and tests implementations against them.

Traditional formal verification improves on this by proving that implementations satisfy specifications.

However, neither approach adequately addresses a critical problem:

The specification itself may be incorrect.

A specification may be:

- contradictory,
- unsatisfiable,
- incomplete,
- over-constrained,
- under-constrained,
- missing important invariants,
- missing important frame conditions,
- or simply describing the wrong behaviour.

Axiomander therefore introduces a distinct development phase:

```
Specification Development
  ↓
Specification Testing
  ↓
Specification Approval
  ↓
Implementation
  ↓
Verification
```

The specification is treated as executable mathematical content that can itself be analysed, tested, challenged and refined before implementation exists.

Core Principle

The objective of specification testing is not to prove that a specification is correct.

The objective is to discover evidence that a specification is wrong.

The process is adversarial.

The tooling should continuously attempt to generate:

- counterexamples,
- contradictory states,
- missing cases,
- unexpected behaviours,
- and vacuous specifications.

before any implementation is written.

Phase 1: Structural Validation

The specification language is first checked for internal consistency.

Examples:

```
requires x > 10  
requires x < 5
```

This is immediately contradictory.

Likewise:

```
ensures result > 0  
ensures result < 0
```

may indicate an impossible postcondition.

The specification checker should attempt to establish:

```
Precondition satisfiable?  
Postcondition satisfiable?  
Invariant satisfiable?  
Frame satisfiable?
```

before proceeding further.

Phase 2: Model Exploration

Axiomander generates abstract program states satisfying the preconditions.

These states are not implementations.

They are logical models.

Example:

```
requires balance >= 0
requires withdrawal <= balance
```

Generated states:

```
balance = 0
withdrawal = 0

balance = 1
withdrawal = 1

balance = 1000
withdrawal = 999
```

The objective is to explore the behavioural surface of the specification.

This frequently reveals missing assumptions.

Phase 3: Counterexample Search

The system actively attempts to violate the intended meaning of the specification while remaining compliant with the written specification.

Example:

```
ensures sorted(result)
```

A model generator may produce:

```
result = []
```

which is technically sorted.

This exposes a missing requirement:

```
ensures permutation(result, input)
```

The specification is therefore refined.

This process continues until obvious loopholes disappear.

Phase 4: Specification Mutation Testing

The system automatically mutates the specification.

Examples:

```
>= becomes >  
forall becomes exists  
and becomes or
```

and then checks whether generated tests detect the mutation.

If changing a clause has no observable effect, the clause may be redundant or meaningless.

This is analogous to mutation testing for programs.

Phase 5: Property-Based Specification Testing

Executable portions of specifications are converted into generators and properties.

Example:

```
ensures forall x in result:  
  x >= 0
```

becomes:

```
@given(...)
def property():
    ...
```

The implementation does not yet exist.

The generated properties are instead run against:

- specification models,
- reference models,
- mock implementations,
- generated witnesses.

This validates that the property itself behaves as expected.

Phase 6: Frame Testing

Frame specifications describe what may change.

Example:

```
frame:
  may_modify Orders[order_id]
  must_not_modify Orders[other_id]
```

Axiomander generates state transitions attempting to violate the frame.

Questions asked include:

```
Can unrelated rows be modified?
Can hidden state be modified?
Can resources be leaked?
Can ownership be duplicated?
```

Frame errors are among the most dangerous classes of specification bugs because they permit unexpected side effects.

Phase 7: Resource and Ownership Testing

Resource specifications should be challenged independently of implementation.

Example:

```
owns lease(job_id)
```

The system should attempt to construct states where:

```
Two workers own the same lease.  
A lease exists without an owner.  
A lease is consumed twice.  
A lease is never released.
```

If such states remain legal under the specification, the ownership model is incomplete.

Phase 8: Invariant Stress Testing

Global invariants are tested against generated operations.

Example:

```
preserves Inventory.nonnegative
```

The system synthesises sequences of abstract operations attempting to violate the invariant.

Examples:

```
reserve  
reserve  
release  
reserve  
cancel
```

or

```
concurrent reserve  
concurrent reserve
```

The goal is to discover whether the invariant is actually strong enough to capture the intended safety property.

Phase 9: Specification Coverage Analysis

The system identifies parts of the state space that are unconstrained.

Examples:

```
ensures result.success
```

provides almost no information.

Coverage analysis attempts to answer:

```
Which fields are unconstrained?  
Which resources are unconstrained?  
Which effects are unconstrained?  
Which outcomes are unconstrained?
```

Large unconstrained regions often indicate weak specifications.

Phase 10: Proof Readiness Analysis

Before implementation begins, the specification is classified.

Each clause is assigned to one of several categories.

Executable

Suitable for testing.

Examples:

```
x > 0
sorted(xs)
len(xs) == len(ys)
```

SMT-Friendly

Likely dischargeable automatically.

Examples:

```
linear arithmetic
sets
maps
bounded quantifiers
```

Theorem-Prover-Oriented

Likely requires interactive proof.

Examples:

```
inductive invariants
logical relations
resource algebra reasoning
refinement proofs
```

Trusted Assumption

External facts.

Examples:

```
database transactions are atomic
payment provider is idempotent
clock drift remains bounded
```

This classification provides an early estimate of verification difficulty.

AI-Assisted Specification Development

The purpose of specification testing is not merely bug finding.

It is also a feedback loop for AI-generated specifications.

The workflow becomes:

```
graph TD; A[Human writes intent] --> B[AI proposes specification]; B --> C[Specification testing]; C --> D[Counterexamples]; D --> E[AI refines specification]; E --> F[Specification testing]; F --> G[Approval]; G --> H[Implementation];
```

Human writes intent
↓
AI proposes specification
↓
Specification testing
↓
Counterexamples
↓
AI refines specification
↓
Specification testing
↓
Approval
↓
Implementation

The implementation is therefore not the first artefact subjected to adversarial scrutiny.

The specification itself is.

This significantly increases confidence that the resulting verified software is actually verifying the intended behaviour rather than an accidentally incomplete specification.